

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

Práctica Experimental Unidad 4 — GA-SUM-05 / PE-U4

Procesamiento Distribuido con Apache Spark: comprobación experimental de la Ley de Amdahl aplicada al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701) [20701]
Unidad: Unidad 4 – Cómputo Paralelo y Distribuido
Código de actividad: GA-SUM-05 / PE-U4
Nivel / Paralelo: Séptimo nivel – Paralelo A
Docente responsable: Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico: 2026–2027 PPA
Semana de entrega: Semana 14
Código del equipo: Equipo C

PFC de referencia elegido

Código del PFC: FUVV
Título del sistema: Sistema de Control de Laboratorios Informáticos (SCLI)

Integrantes del equipo

Integrante	PFC de origen	Rol asumido
Freddy Farinango	FUVV	Coordinación del equipo, despliegue del clúster Spark Standalone, protocolo experimental y redacción $\LaTeX$
Jeremy Gaibor	FUVV	Implementación de las transformaciones T1–T5 en pandas y PySpark, medición de tiempos y redacción $\LaTeX$
Iván Villamarín	FUVV	Análisis cuantitativo, generación de figuras, verificación de equivalencia y redacción $\LaTeX$

La redacción, estructuración y compilación del documento $\LaTeX$ fue una labor colaborativa de los tres integrantes.

Justificación técnica de la elección del PFC (dominio FUVV). El dominio FUVV, correspondiente al Sistema de Control de Laboratorios Informáticos (SCLI), se eligió porque es el dominio que con mayor naturalidad genera volúmenes de datos que justifican un motor distribuido. Cada laboratorio de la facultad produce, de manera continua, solicitudes de reserva con su ciclo de estados (pendiente, aprobada, rechazada, cancelada, finalizada), registros de asistencia y de uso de equipos, y eventos de monitoreo por estación de trabajo. Acumulados a lo largo de varios períodos académicos y proyectados a decenas de laboratorios con cientos de equipos, estos flujos superan con facilidad el millón de registros anuales, un volumen para el cual el procesamiento en un solo proceso deja de ser la única alternativa razonable y en el que resulta pertinente evaluar empíricamente cuándo un motor distribuido como Apache Spark aporta valor y cuándo su sobrecarga lo penaliza. La decisión de negocio que se apoya en el pipeline implementado es la *planificación de capacidad y priorización de reservas*: a partir del filtrado de solicitudes aprobadas de alta ocupación (T1), de los agregados de demanda por laboratorio (T2), del cruce con la dimensión de laboratorios y sus capacidades (T3), del puntaje determinista de prioridad (T4) y del ranking de mayor demanda (T5), la administración puede decidir qué laboratorios ampliar, en qué franjas concentrar mantenimiento y qué solicitudes priorizar cuando la capacidad es insuficiente.

Repositorio del proyecto: <https://github.com/ffarinangog2/pe-u4-spark-equipo-c>

Procesamiento Distribuido con Apache Spark: comprobación experimental de la Ley de Amdahl aplicada al dominio FUVV de reserva de laboratorios informáticos

Freddy Farinango, Jeremy Gaibor e Iván Villamarín

Carrera de Ingeniería de Software, Facultad de Ciencias de la Computación

Universidad Técnica Estatal de Quevedo, Quevedo, Ecuador

Aplicaciones Distribuidas (ISR-701) — Docente: Gleiston C. Guerrero-Ulloa, M.Sc. — Período 2026–2027 PPA

RESUMEN

Esta práctica comprueba experimentalmente la Ley de Amdahl mediante la implementación y medición de un pipeline de cinco transformaciones de datos (T1–T5) ejecutado de forma secuencial con pandas y de forma distribuida con PySpark 3.5.0 sobre un clúster Spark Standalone real, utilizando un conjunto de datos sintético y reproducible de 500 000 solicitudes de reserva de laboratorios del dominio FUVV. Cada transformación se midió con `time.perf_counter()` en cinco repeticiones tras una ejecución de calentamiento descartada, reportando la mediana. Con cuatro executors, PySpark superó a pandas solo en las transformaciones intensivas por fila (T4, con speedup de 1,251 9, y T5, con 1,124 4), mientras que en T1, T2 y T3 el speedup fue inferior a la unidad debido a la sobrecarga de planificación y *shuffle* del motor. El escalado de T3 con 1, 2 y 4 executors arrojó speedups de 1,000 0, 1,921 7 y 1,572 7; aplicando la forma inversa de la Ley de Amdahl (métrica de Karp-Flatt) se obtuvo una fracción no escalable observada de $f = 0,5145$, que impone un límite teórico máximo de $S_{\text{máx}} = 1,9437$ alcanzable con infinitos procesadores. Los resultados confirman cuantitativamente que el límite de la paralelización lo fija la fracción serial y que, para el volumen evaluado, el motor distribuido solo es rentable en transformaciones con cómputo por registro suficientemente costoso.

PALABRAS CLAVE

Ley de Amdahl; Apache Spark; PySpark; pandas; speedup; fracción serial; eficiencia paralela; Gustafson-Barsis; procesamiento distribuido; benchmarking.

I. INTRODUCCIÓN

El cómputo paralelo y distribuido constituye hoy la vía dominante para procesar volúmenes de datos que exceden la capacidad de un solo proceso, pero su adopción no es gratuita: todo motor distribuido introduce costos de planificación, serialización, transferencia y coordinación que compiten con la ganancia del paralelismo. La Ley de Amdahl formalizó tempranamente este compromiso al demostrar que el speedup alcanzable por un programa está acotado por su fracción estrictamente serial, independientemente del número de procesadores disponibles [1]. Décadas después, la reformulación de Gustafson-Barsis matizó esa cota para escenarios en los que la carga de trabajo crece con los recursos [2], y la métrica de Karp-Flatt propuso estimar empíricamente la fracción serial a partir de mediciones reales [3].

Esta práctica experimental cierra el recorrido conceptual de la Unidad 4 de la asignatura Aplicaciones Distribuidas conectando cuatro capas: el modelo teórico de límites de paralelización [1], [2]; el paradigma MapReduce que popularizó el procesamiento por lotes a gran escala [4]; el motor unificado Apache Spark, que superó a MapReduce para cargas iterativas mediante el modelo RDD en memoria [5]; y los modelos de computación en la nube sobre los que estos motores se despliegan en producción [6].

El experimento se ancla en un dominio de aplicación concreto: el PFC de referencia FUVV, Sistema de Control de Laboratorios Informáticos (SCLI). El pipeline implementado no es un ejercicio abstracto sino la versión analítica de una necesidad real del dominio: filtrar las solicitudes aprobadas de alta ocupación, agregar la demanda por laboratorio, cruzarla con la capacidad instalada, puntuar la prioridad de cada solicitud y producir el ranking de mayor demanda que alimenta las decisiones de planificación de capacidad. Medir ese pipeline en dos motores y confrontar las mediciones con el modelo formal permite responder una pregunta de ingeniería que el PFC deberá enfrentar: a partir de qué volumen, y con cuántos recursos, conviene distribuir el procesamiento.

El objetivo general es comprobar experimentalmente la Ley de Amdahl implementando y midiendo el rendimiento de un pipeline distribuido de datos con Apache Spark sobre un conjunto de 500 000 registros del dominio FUVV, comparándolo con el procesamiento secuencial equivalente en pandas. Los objetivos específicos son: (i) implementar las mismas cinco transformaciones con pandas y con PySpark; (ii) medir los tiempos con precisión de microsegundos, con cinco repeticiones por transformación y reporte de la mediana; (iii) calcular el speedup experimental y compararlo con el speedup teórico de Amdahl; y (iv) determinar la fracción serial observada y el límite práctico de paralelización.

El resto del documento se organiza así: la Sección II desarrolla el fundamento teórico de la unidad; la Sección III describe el procedimiento aplicado, el entorno, el conjunto de datos y el protocolo de medición; la Sección IV presenta los resultados experimentales con el trabajo matemático explícito del análisis de Amdahl; la Sección V discute críticamente los hallazgos, incluida la anomalía de escalado entre dos y cuatro executors; la Sección VI responde las preguntas obligatorias del Anexo A de la guía; y la Sección VII recoge las conclusiones individuales de los integrantes.

II. FUNDAMENTO TEÓRICO

II-A. Fundamentos de paralelismo y Ley de Amdahl

El paralelismo busca reducir el tiempo de resolución de un problema repartiendo trabajo entre varias unidades de procesamiento, y se manifiesta en dos formas complementarias. El *paralelismo de datos* aplica la misma operación simultáneamente sobre particiones disjuntas de un conjunto de datos, y es el modelo natural de los motores de procesamiento masivo como MapReduce y Spark, en los que cada tarea procesa una partición del conjunto [4], [5]. El *paralelismo de tareas*, en cambio, ejecuta simultáneamente operaciones distintas, posiblemente sobre los mismos datos, y aparece en Spark en la ejecución concurrente de etapas independientes del grafo de ejecución [5], [7]. En ambos casos la ganancia está limitada por la porción del trabajo que no puede repartirse.

Amdahl formuló en 1967 el argumento cuantitativo clásico sobre ese límite [1]. Sea T_1 el tiempo de ejecución con una unidad de procesamiento, p la fracción del tiempo que corresponde a trabajo paralelizable y $(1-p)$ la fracción estrictamente serial. Con N unidades, la parte paralelizable idealmente se reduce a pT_1/N , mientras que la serial permanece invariante, de modo que $T_N = (1-p)T_1 + \frac{p}{N}T_1$. El speedup, definido como $S(N) = T_1/T_N$, resulta

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad (1)$$

que es la Ley de Amdahl [1], [3]. Tomando el límite cuando $N \rightarrow \infty$, el término p/N se anula y el speedup máximo queda fijado exclusivamente por la fracción serial:

$$S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}. \quad (2)$$

La consecuencia práctica de (2) es severa: con apenas un 10 % de código serial ($p = 0,9$), ningún número de procesadores puede superar un speedup de 10.

Gustafson observó que en la práctica científica el tamaño del problema crece con los recursos disponibles: no se resuelve el mismo problema más rápido, sino un problema mayor en el mismo tiempo [2]. Manteniendo fija la fracción serial del tiempo de ejecución paralelo, el speedup escalado de Gustafson-Barsis es

$$S_G(N) = N - (1-p)(N-1), \quad (3)$$

una recta creciente en N que no presenta la saturación de (1) y que resulta el modelo adecuado cuando la carga escala con el clúster [2], [3].

Despejando p de (1) para una medición experimental con N unidades y speedup observado S , se obtiene la fracción serial implícita, que es la métrica de Karp-Flatt en su forma equivalente [3]:

$$f = 1-p = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (4)$$

Finalmente, la eficiencia del paralelismo mide qué proporción de la capacidad agregada se aprovecha realmente:

$$E(N) = \frac{S(N)}{N}. \quad (5)$$

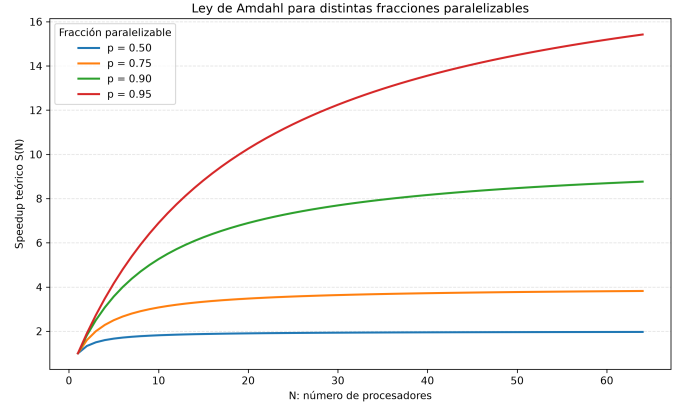


Figura 1. Speedup teórico de Amdahl según (1) frente al número de procesadores N para fracciones paralelizables $p = 0,5; 0,75; 0,9$ y $0,95$. Cada curva satura en la asíntota $1/(1-p)$ dada por (2). Figura generada con matplotlib por el script `src/graficas.py` del repositorio.

La eficiencia (5) tiene su propia lectura asintótica: como $S(N)$ satura en $1/(1-p)$, la eficiencia $E(N) = S(N)/N$ tiende a cero cuando N crece, de modo que todo sistema de talla fija termina desperdiciando la mayor parte de su capacidad agregada si se sobredimensiona [1], [3]. De ahí se deriva una regla de diseño que esta práctica comprueba empíricamente: el número óptimo de unidades de procesamiento no es el máximo disponible, sino el mayor N para el cual la eficiencia se mantiene por encima de un umbral aceptable, típicamente entre el 70 % y el 90 %. Karp y Flatt añaden el uso diagnóstico de (4): si al medir con valores crecientes de N la fracción serial experimental permanece constante, la pérdida de eficiencia se debe al paralelismo limitado del algoritmo; si f crece con N , la causa es la sobrecarga creciente del sistema (planificación, comunicación, contención), un síntoma que exige optimizar la plataforma y no reescribir el algoritmo [3].

Es indispensable una advertencia metodológica sobre (4): la magnitud despejada a partir de mediciones no es la fracción serial *algorítmica*, sino la fracción *no escalable observada*, que agrega a la porción serial del algoritmo toda la sobrecarga de planificación, serialización, comunicación y *shuffle* del motor de ejecución [3], [8]. Confundir ambas conduce a subestimar sistemáticamente el potencial de paralelización del algoritmo puro. La Fig. 1 representa (1) para fracciones paralelizables $p = 0,5; 0,75; 0,9$ y $0,95$, y evidencia gráficamente la saturación que impone (2): las cuatro curvas se aplanan hacia asíntotas de valor 2, 4, 10 y 20 respectivamente, de modo que añadir procesadores más allá de cierto punto produce ganancias marginales despreciables.

II-B. Paradigma MapReduce y Apache Hadoop

MapReduce es el modelo de programación propuesto por Dean y Ghemawat en Google para procesar grandes conjuntos de datos sobre clústeres de máquinas convencionales, ocultando al programador los detalles de particionamiento, planificación, tolerancia a fallos y comunicación [4]. Formalmente, el usuario define dos funciones. La función $map : (k_1, v_1) \rightarrow list(k_2, v_2)$ procesa cada par clave-valor de entrada y emite pares intermedios; la función $reduce : (k_2, list(v_2)) \rightarrow$

$list(v_3)$ recibe todos los valores intermedios asociados a una misma clave y los combina en el resultado final [4], [9]. El flujo completo de ejecución consta de cinco fases: *split*, en la que la entrada se divide en fragmentos; *map*, en la que cada fragmento se procesa en paralelo; *shuffle*, en la que los pares intermedios se redistribuyen por clave entre los nodos; *sort*, en la que cada nodo ordena las claves recibidas; y *reduce*, en la que se agregan los valores por clave [4].

Apache Hadoop es la implementación de código abierto de este modelo y se apoya en dos subsistemas. HDFS es un sistema de archivos distribuido que divide cada archivo en bloques de gran tamaño y los replica, por defecto tres veces, entre nodos de datos coordinados por un nodo de nombres que mantiene los metadatos; la replicación proporciona a la vez tolerancia a fallos de disco y de nodo y oportunidades de *localidad de datos*, es decir, de planificar cada tarea map en un nodo que ya alberga una réplica de su bloque, evitando transferirlo por la red [9], [10]. YARN, por su parte, separa la gestión de recursos de la lógica de las aplicaciones: un gestor de recursos global negocia contenedores de CPU y memoria con los gestores de nodo, y cada aplicación ejecuta su propio maestro que solicita contenedores y supervisa sus tareas, lo que permite que MapReduce, Spark y otros motores convivan en el mismo clúster [11]. Frente al procesamiento tradicional en un solo servidor, el modelo ofrece escalabilidad horizontal casi lineal para cargas por lotes, tolerancia a fallos mediante reejecución determinista de tareas fallidas y ocultamiento completo de la distribución al programador [4], [10], [9].

Su limitación central aparece en el procesamiento *iterativo*: cada iteración es un trabajo MapReduce independiente que materializa sus resultados en disco, de modo que algoritmos que reutilizan datos entre iteraciones pagan en cada ciclo el costo completo de lectura y escritura en HDFS [5], [9]. La evidencia empírica publicada confirma esta brecha: en el estudio sistemático de Ahmed *et al.* con la suite HiBench, Spark superó a Hadoop en la mayoría de las cargas evaluadas, con ventajas especialmente amplias en cargas iterativas de aprendizaje automático, aunque la magnitud de la ventaja depende de la carga y de la configuración del clúster [8]. Este resultado arbitrado, y no material promocional, es el que sustenta en este informe toda comparación de rendimiento entre ambos motores [8], [5].

II-C. Apache Spark: arquitectura y modelo de programación

Apache Spark es un motor unificado de procesamiento de datos a gran escala cuya arquitectura distingue tres componentes [5], [7]: el *driver*, proceso que ejecuta el programa del usuario, construye el plan de ejecución y coordina el trabajo; el *cluster manager* (Standalone, YARN, Mesos o Kubernetes), que asigna recursos físicos; y los *executors*, procesos JVM distribuidos que ejecutan tareas sobre particiones de datos y cachean resultados en memoria. En esta práctica se empleó el gestor Standalone propio de Spark con executors explícitamente registrados, lo que permite controlar N en el experimento.

La abstracción fundacional del motor es el RDD (*Resilient Distributed Dataset*): una colección de solo lectura, particionada entre los nodos, que registra el *lineage* de transformaciones

gruesas con que fue construida [12], [5]. La tolerancia a fallos no requiere replicar los datos: ante la pérdida de una partición, el motor la reconstruye reaplicando el lineage sobre los datos de origen, un mecanismo más barato que la replicación de HDFS para cómputo en memoria [5]. Las transformaciones se clasifican en *estrechas*, en las que cada partición de salida depende de una sola partición de entrada (filtros, proyecciones, columnas derivadas), y *anchas*, en las que una partición de salida depende de muchas de entrada y exige redistribuir datos por la red mediante shuffle (agrupaciones, joins particionados, ordenamientos globales) [12], [13]; la distinción es central para este experimento, porque T1 y T4 son estrechas mientras que T2, T3 y T5 son anchas, y el costo del shuffle es precisamente el componente de sobrecarga que la fracción no escalable observada captura. Sobre los RDD se construyen las API de DataFrames, colecciones con esquema relacional explícito, y Datasets, su variante con tipado estático en Scala y Java; ambas delegan en el optimizador Catalyst, que aplica reglas de reescritura al plan lógico (poda de columnas, empuje de predicados hacia la fuente, reordenación y elección de estrategia de join) antes de generar el plan físico [7]. Precisamente por la acción de Catalyst, las transformaciones sobre DataFrames no se ejecutan en el orden en que se escriben: el plan lógico declarado se reescribe antes de ejecutarse, y por ello la Spark UI muestra un DAG que no coincide literalmente con el texto del programa [7].

El modelo de ejecución es perezoso: las *transformaciones* (filter, select, groupBy, join) solo declaran el plan, y únicamente las *acciones* (count, collect, write) disparan la ejecución [5], [7]. Al invocarse una acción, el driver compila el lineage en un DAG de *stages* delimitados por las fronteras de shuffle; cada stage se descompone en *tasks*, una por partición, que el planificador distribuye entre los executors [5], [7]. Esta materialización explícita es la razón metodológica por la que en este experimento cada medición de PySpark fuerza una acción: sin ella se mediría solo la construcción del plan. En cuanto a rendimiento comparado, Spark evita la materialización a disco entre etapas que impone MapReduce, y la evidencia empírica de benchmarking le atribuye ventajas consistentes, especialmente en cargas iterativas [8], [5].

II-D. Computación en la nube: modelos y proveedores

El NIST define la computación en la nube como un modelo que habilita acceso ubicuo, conveniente y bajo demanda a un conjunto compartido de recursos de cómputo configurables, aprovisionables con mínimo esfuerzo de gestión, y enumera cinco características esenciales: autoservicio bajo demanda, amplio acceso por red, agrupación de recursos, elasticidad rápida y servicio medido [6]. Sobre esa base distingue los modelos de servicio IaaS, PaaS y SaaS, según el nivel de la pila que gestiona el proveedor [6]; la industria añadió posteriormente FaaS, en el que la unidad de despliegue es la función y el proveedor gestiona todo el ciclo de ejecución, extensión natural de la elasticidad y del pago por uso que la literatura fundacional de la nube identificó como sus rasgos económicos distintivos [14].

Las cinco características esenciales merecen glosa porque explican por qué los motores distribuidos migraron a la nube.

El *autoservicio bajo demanda* elimina la negociación humana del aprovisionamiento; el *amplio acceso por red* desacopla el consumo del lugar; la *agrupación de recursos* permite al proveedor multiplexar clientes sobre el mismo hardware con economías de escala; la *elasticidad rápida* habilita crecer y decrecer el clúster al ritmo de la carga, condición necesaria para que el régimen de Gustafson sea alcanzable en la práctica; y el *servicio medido* convierte el cómputo en costo variable, lo que hace del sobredimensionamiento un despilfarro contable inmediato [6]. En los modelos de servicio, IaaS entrega máquinas virtuales y redes sobre las que el usuario instala el motor; PaaS entrega el motor gestionado; SaaS entrega la aplicación final; y FaaS entrega ejecución de funciones dirigida por eventos sin gestión de servidores [6], [15], [14].

Para el procesamiento distribuido, los tres grandes proveedores ofrecen servicios gestionados equivalentes que despliegan clústeres Spark/Hadoop sin administración manual: EMR en AWS, HDInsight en Azure y Dataproc en GCP. Los tres siguen el patrón PaaS sobre infraestructura elástica: el usuario declara tamaño y tipo de nodos y el proveedor gestiona aprovisionamiento, parcheo y escalado, diferenciándose principalmente en el ecosistema que los rodea (almacenamiento de objetos, catálogos de datos y servicios de análisis integrados de cada nube) y en la granularidad de la facturación [6], [15]. El dimensionamiento de recursos para flujos de trabajo basados en DAG, como los de Spark, es en sí mismo un problema de optimización costo-rendimiento: la literatura arbitrada propone recomendadores que, dado un flujo y un objetivo de plazo, seleccionan tipo y número de instancias minimizando el costo monetario, precisamente porque añadir nodos más allá del límite de Amdahl solo incrementa la factura sin reducir el tiempo [16], [1].

En cuanto a arquitecturas de referencia, la arquitectura *Lambda* combina una capa por lotes, que reprocesa periódicamente el histórico completo, con una capa de velocidad que procesa el flujo reciente, unificando ambas vistas en la capa de servicio; la arquitectura *Kappa* elimina la capa por lotes y trata todo como flujo reprocesable desde un log inmutable [17]. Para el dominio FUVV, una *Kappa* sobre Spark Structured Streaming resulta natural, como se argumenta en la Sección VI.

III. PROCEDIMIENTO APLICADO

III-A. Entorno de ejecución y clúster Spark Standalone

El experimento se ejecutó en una estación de trabajo local con Python 3.12 y las versiones fijadas en `requirements.txt`: PySpark 3.5.0, pandas 2.3.3, NumPy 1.26.4 y matplotlib 3.11.1. A diferencia del modo `local[N]`, que simula paralelismo con hilos dentro de un solo proceso JVM, se levantó un clúster *Spark Standalone real*, con un proceso master en `spark://127.0.0.1:7077`, un worker registrado y N procesos executor JVM independientes de un core y 512 MiB de memoria cada uno. El código de creación de la sesión (Listado 1) exige que los N executors estén efectivamente registrados antes de medir, verifica mediante el `StatusTracker` que el número de executors (excluido el driver) coincida con el solicitado, aborta si el master resultara

Tabla I
VERSIONES DE SOFTWARE Y CONFIGURACIÓN EFECTIVA DEL CLÚSTER.

Elemento	Valor declarado
Python	3.12
PySpark	3.5.0
pandas	2.3.3
NumPy	1.26.4
matplotlib	3.11.1
Gestor de clúster	Spark Standalone
<code>spark.master</code>	<code>spark://127.0.0.1:7077</code>
<code>spark.executor.instances</code>	1, 2 y 4
<code>spark.executor.cores</code>	1
<code>spark.executor.memory</code>	512 MiB
<code>spark.cores.max</code>	igual a N
<code>spark.sql.shuffle.partitions</code>	igual a N
<code>spark.default.parallelism</code>	igual a N
Reloj de medición	<code>time.perf_counter()</code>
Repeticiones	5 + 1 calentamiento
Estadístico central	Mediana

`local[N]`, y documenta la configuración efectiva obtenida con `spark.sparkContext.getConf().getAll()`, tal como exige la guía. Los parámetros `spark.sql.shuffle.partitions` y `spark.default.parallelism` se fijaron en N para que el número de particiones de shuffle acompañe al número de executors. La Tabla I declara las versiones y la configuración efectiva del experimento, requisito de la guía para la trazabilidad de las mediciones. Cada configuración de N executors empleó una sesión independiente, creada y destruida en orden, de modo que ninguna medición hereda cachés ni estado de una configuración anterior.

```
def crear_spark_session(executors: int) -> SparkSession:
    """Conecta con Spark Standalone y exige N executors
    registrados."""
    spark = (
        SparkSession.builder.master(SPARK_MASTER) #
        spark://127.0.0.1:7077
        .appName(f"PE-U4-benchmark-executors-{executors}")
        .config("spark.executor.instances", str(executors))
        .config("spark.executor.cores", "1")
        .config("spark.cores.max", str(executors))
        .config("spark.executor.memory", "512m")
        .config("spark.sql.shuffle.partitions",
            str(executors))
        .config("spark.default.parallelism", str(executors))
        .getOrCreate()
    )
    # Espera activa: N executors registrados (el
    # StatusTracker incluye al driver)
    limite = time.monotonic() + 60
    while time.monotonic() < limite:
        registrados = max(0, len(
            spark.sparkContext._jsc.sc()
            .statusTracker().getExecutorInfos() - 1))
        if registrados == executors:
            break
        time.sleep(1)
    else:
        spark.stop()
        raise RuntimeError("Spark Standalone no registro
        los executors")
    conf = dict(spark.sparkContext.getConf().getAll())
    if conf.get("spark.master", "").startswith("local"):
        spark.stop()
        raise RuntimeError("master local[N] no es un
        cluster real")
    return spark
```

Listado 1. Creación y verificación de la sesión Spark sobre el clúster Standalone (extracto de `src/medicion.py`).

III-B. Conjunto de datos: declaración y documentación

El dominio FUVV corresponde a un sistema académico interno cuyos datos operacionales contienen información personal de estudiantes y docentes y no están publicados; tras revisar Kaggle y los portales de datos abiertos de Ecuador no se halló un conjunto real público con el esquema de solicitudes de reserva de laboratorios y el volumen exigido. En consecuencia, y acogiéndose a la excepción prevista en la guía, **se declara explícitamente que el conjunto de datos es sintético**, generado con el script propio `src/generar_dataset.py` con semilla fija 20260804, publicado en el repositorio junto con su comando de verificación (`python src/generar_dataset.py -verificar`). Dos regeneraciones consecutivas con la misma semilla produjeron huellas SHA-256 idénticas, documentadas en `data/README_dataset.md`, lo que garantiza la reproducibilidad exacta del insumo. La decisión se justifica porque permite controlar el volumen, conservar el esquema conceptual del PFC y evitar el uso de información personal real.

El conjunto consta de una tabla de hechos de 500 000 solicitudes de reserva con 14 columnas (identificadores UUID, fechas, horas, participantes, motivo, estado y marcas temporales) y una tabla dimensional de 40 laboratorios con 10 columnas, relacionadas 1:N mediante `laboratorio_id`. La Tabla II documenta fuente, URL, licencia, fecha, registros, columnas y tamaño en disco. Ambas tablas se cargaron en pandas y en DataFrames de PySpark con esquema explícito, verificando en los dos motores el conteo de 500 000 y 40 registros y la coincidencia del esquema inferido.

III-C. Implementación secuencial con pandas

Las cinco transformaciones exigidas se implementaron en `src/transformaciones_pandas.py` como funciones puras sin efectos de carga, de modo que la medición pudiera controlarse externamente:

- **T1 — filtrado compuesto y selección:** solicitudes con `estado = APROBADA`, `numero_participantes` ≥ 25 y `fecha_reserva` en el rango inclusivo 2024-07-01 a 2025-06-30, proyectando siete columnas.
- **T2 — agrupación con tres agregaciones:** `groupby` por `laboratorio_id` con conteo de solicitudes, promedio y máximo de participantes.
- **T3 — join: inner join N:1** de las 500 000 solicitudes con la dimensión de 40 laboratorios mediante `laboratorio_id`, validado como `many_to_one`.
- **T4 — columna derivada compleja:** puntaje determinista de prioridad que combina participantes por duración en minutos con bonos por ocupación ($\geq 90\%$: 1000; $\geq 75\%$: 500), anticipación (≤ 7 días: 300), fin de semana (200) y estado aprobado (100).
- **T5 — ordenamiento y top-N:** las 20 solicitudes de mayor demanda con orden estable por participantes y duración descendentes y fecha e identificador ascendentes.

El Listado 2 muestra el núcleo de T4, la transformación más costosa del pipeline. Cada transformación se ejecutó de

manera independiente, sin reutilizar resultados intermedios de la anterior (T4 recibe el join reconstruido, no el resultado medido de T3), para que los tiempos individuales fueran comparables, y sus salidas se persistieron en formato CSV en `data/pandas/`. Las funciones validan explícitamente el esquema de entrada y generan errores descriptivos ante columnas ausentes, y todas las conversiones temporales se realizan con `errors="raise"` para que ninguna fila malformada pase inadvertida; la implementación evita bucles de Python en favor de operaciones vectorizadas (máscaras, `mask`, aritmética de series), que es la condición para que la comparación con Spark enfrente a pandas en su mejor forma y no a una implementación ingenua. El ordenamiento de T5 emplea el algoritmo estable `mergesort` para garantizar el desempate determinista que la verificación de equivalencia exige.

```
inicio = pd.to_timedelta(resultado["hora_inicio"],
    errors="raise")
fin = pd.to_timedelta(resultado["hora_fin"], errors="raise")
duracion_minutos = ((fin - inicio).dt.total_seconds() //
    60).astype("int64")

ocupacion_por_cien = participantes * 100
bono_ocupacion = pd.Series(0, index=resultado.index,
    dtype="int64")
bono_ocupacion = bono_ocupacion.mask(ocupacion_por_cien >=
    capacidad * 75, 500)
bono_ocupacion = bono_ocupacion.mask(ocupacion_por_cien >=
    capacidad * 90, 1000)

bono_antelacion = antelacion_dias.le(7).astype("int64") *
    300
bono_fin_semana =
    fecha_reserva.dt.dayofweek.ge(5).astype("int64") * 200
bono_aprobada =
    resultado["estado"].eq("APROBADA").astype("int64") *
    100

resultado["puntaje_prioridad"] = (
    participantes * duracion_minutos
    + bono_ocupacion + bono_antelacion
    + bono_fin_semana + bono_aprobada
).astype("int64")
```

Listado 2. Núcleo de la columna derivada T4 en pandas (extracto de `src/transformaciones_pandas.py`).

III-D. Implementación distribuida con PySpark

Las mismas cinco transformaciones se implementaron sobre DataFrames de PySpark en `src/transformaciones_spark.py`, cuidando la equivalencia semántica exacta con pandas: el rango de fechas de T1 es inclusivo en ambos extremos; T4 reproduce la normalización UTC de fechas de pandas extrayendo la subcadena ISO YYYY-MM-DD para neutralizar el efecto de `spark.sql.session.timeZone`; y T5 aplica los mismos cuatro criterios de ordenación con desempate estable. El Listado 3 muestra el núcleo equivalente de T4. Dado el modelo de evaluación perezosa descrito en la Sección II-C, cada medición forzó la materialización con una acción explícita (`count()` sobre el DataFrame resultante), de modo que el tiempo registrado incluye la ejecución real del DAG y no solo su construcción. Para T3, la transformación con `shuffle` designada para el análisis de escalabilidad, se midió adicionalmente con 1, 2 y 4 executors, creando para cada configuración una sesión independiente verificada según el Listado 1. Las salidas de las cinco transformaciones se escribieron en `data/spark/`.

Tabla II
DOCUMENTACIÓN DEL CONJUNTO DE DATOS SINTÉTICO DECLARADO (DOMINIO FUVV).

Archivo	Fuente	Lic.	Fecha gen.	Registros	Cols.	Tamaño (B)
solicitudes_reserva.csv	Sintética (generar_dataset.py, semilla 20260804)	MIT	2026-08-05	500 000	14	176 712 866
laboratorios.csv	Sintética (generar_dataset.py, semilla 20260804)	MIT	2026-08-05	40	10	8 754

URL permanente del script generador y del conjunto: <https://github.com/ffarinangog2/pe-u4-spark-equipo-c>. Los conteos excluyen la fila de encabezado; los tamaños y las huellas SHA-256 se documentan en data/README_dataset.md.

Tres decisiones de implementación merecen justificación por su efecto sobre las mediciones. Primera: no se aplicó broadcast al DataFrame de laboratorios pese a que sus 40 filas lo harían candidato natural, porque el objetivo experimental de T3 es observar el comportamiento de una transformación con shuffle frente al número de executors; forzar un join por difusión habría eliminado precisamente el fenómeno bajo estudio y habría hecho inaplicable el análisis de Amdahl. Segunda: no se empleó `cache()` ni `persist()` en ningún punto del pipeline, de modo que cada medición parte de la lectura del origen y ninguna transformación se beneficia del trabajo de la anterior. Tercera: la lectura del CSV se realizó con esquema explícito en lugar de inferencia automática, porque la inferencia exige una pasada adicional completa sobre el archivo y habría contaminado los tiempos con un costo ajeno a la transformación medida. En conjunto, las tres decisiones sesgan el experimento en contra de PySpark en términos absolutos, lo que refuerza la validez de las conclusiones cuando el speedup sí supera la unidad.

El notebook `notebooks/PE_U4_pipeline_spark.ipynb` conserva las salidas de todas las celdas y se exporta también como HTML.

```

duracion_minutos = _duracion_en_minutos("hora_inicio",
    "hora_fin")
antelacion_dias = F.datediff(fecha_reserva, fecha_creacion)

ocupacion_por_cien = F.col("numero_participantes") *
    F.lit(100)
bono_ocupacion = (
    F.when(ocupacion_por_cien >= F.col("capacidad") *
        F.lit(90), F.lit(1000))
    .when(ocupacion_por_cien >= F.col("capacidad") *
        F.lit(75), F.lit(500))
    .otherwise(F.lit(0)))
bono_antelacion = F.when(antelacion_dias <= F.lit(7),
    F.lit(300)).otherwise(F.lit(0))
bono_fin_semana = F.when(
    F.dayofweek(fecha_reserva).isin(1, 7),
    F.lit(200)).otherwise(F.lit(0))
bono_aprobada = F.when(
    F.col("estado") == F.lit("APROBADA"),
    F.lit(100)).otherwise(F.lit(0))

puntaje = (F.col("numero_participantes") * duracion_minutos
    + bono_ocupacion + bono_antelacion
    + bono_fin_semana + bono_aprobada).cast("long")

return
    solicitudes_con_laboratorio.withColumn("puntaje_prioridad",
        puntaje)

```

Listado 3. Núcleo de la columna derivada T4 en PySpark (extracto de `src/transformaciones_spark.py`).

III-E. Protocolo de medición

Todas las mediciones emplearon `time.perf_counter()`, el reloj monotónico de mayor resolución disponible en Python, con precisión de microsegundos. Para cada transformación y configuración se ejecutó primero una *ejecución de*

calentamiento, cuyo tiempo se registró pero **se descartó explícitamente** del cálculo, con el fin de absorber efectos de compilación JIT de la JVM, poblado de cachés y primera materialización de planes; a continuación se realizaron **cinco repeticiones medidas**, reportando la **mediana** como estadístico central por su robustez frente a valores atípicos. El Listado 4 muestra el núcleo del protocolo. Los sesenta tiempos crudos (cinco repeticiones por cada una de las doce series motor–transformación–configuración) se publican en `resultados/tiempos_crudos.csv`, y las medianas y speedups en `resultados/tiempos_resumen.csv`, lo que permite reconstruir cada cifra de este informe.

```

# Calentamiento: se ejecuta, se registra y se DESCARTA del
# calculo
inicio_calentamiento = time.perf_counter()
resultado_calentamiento = operacion()
materializar(resultado_calentamiento) # accion count() en
PySpark
tiempo_calentamiento = time.perf_counter() -
    inicio_calentamiento
registro_calentamientos.append(...)
del resultado_calentamiento

# Cinco repeticiones medidas
for repeticion in range(1, repeticiones + 1):
    inicio = time.perf_counter()
    resultado = operacion()
    materializar(resultado)
    tiempo_segundos = time.perf_counter() - inicio
    mediciones.append(Medicion(..., tiempo_segundos))

```

```

# Estadístico central: mediana de las repeticiones
homogeneas
mediana_segundos = statistics.median(tiempos)

```

Listado 4. Núcleo del protocolo de medición: calentamiento descartado, cinco repeticiones y mediana (extracto de `src/medicion.py`).

La elección de `time.perf_counter()` frente a otros relojes de la biblioteca estándar responde a sus propiedades: es monotónico, por lo que no le afectan los ajustes del reloj del sistema durante la medición, y ofrece la mayor resolución disponible en la plataforma. La elección de la mediana frente a la media responde a la naturaleza de las perturbaciones: los valores atípicos de una serie de medición en un sistema con recolector de basura, JIT y procesos concurrentes son casi siempre unilaterales hacia arriba, y la media los propaga mientras que la mediana los descarta. Ambas decisiones, junto con el calentamiento descartado y la ejecución independiente de cada transformación, conforman el protocolo único aplicado idénticamente a los dos motores, condición necesaria para que los speedups sean comparables.

III-F. Verificación de equivalencia de resultados

La comparación de tiempos solo es válida si ambos motores computan el mismo resultado. La equivalencia pandas/PySpark se verificó para las cinco transformaciones mediante comparación de cardinalidad y de agregados de control (sumas,

Tabla III
VERIFICACIÓN DE EQUIVALENCIA PANDAS/PYSPARK POR TRANSFORMACIÓN.

Transf.	Card. pandas	Card. PySpark	Agregados
T1	33 117	33 117	Idénticos
T2	40	40	Idénticos
T3	500 000	500 000	Idénticos
T4	500 000	500 000	Idénticos
T5	20	20	Idénticos

promedios y conteos por clave). Los agregados de control se eligieron por transformación: para T1, el conteo de filas filtradas y la suma de participantes seleccionados; para T2, la igualdad exacta de las 40 filas agregadas; para T3, la cardinalidad del join y los conteos por `laboratorio_id`; para T4, la suma total de `puntaje_prioridad`, sensible a cualquier divergencia en fechas, duraciones o bonos; y para T5, la identidad ordenada de los 20 registros. La Tabla III resume la verificación: las cardinalidades coinciden exactamente en los cinco casos, los agregados por `laboratorio_id` de T2 son idénticos en las 40 claves, la suma de `puntaje_prioridad` de T4 coincide entre motores y los 20 registros de T5 son los mismos y en el mismo orden.

III-G. Repositorio y reproducibilidad de extremo a extremo

Todo el trabajo reside en el repositorio público `pe-u4-spark-equipo-c`, cuya estructura sigue exactamente la exigida por la guía: `notebooks/` con el notebook ejecutado y su exportación HTML; `src/` con los módulos de transformaciones, medición, gráficas y generación del conjunto de datos; `data/` con la documentación del conjunto y las salidas persistidas de cada motor; `resultados/` con los tiempos crudos, el resumen de medianas y las figuras a 300 DPI; `evidencia/` con las capturas de la Spark UI y del master del clúster; y `docs/` con este documento fuente, su PDF compilado y la bibliografía. El `README.md` documenta las instrucciones exactas de reproducción: creación del entorno con las versiones fijadas de `requirements.txt`, regeneración del conjunto con su semilla declarada, arranque del master y el worker Standalone, ejecución del protocolo de medición, generación de las figuras y compilación del informe con la secuencia `pdflatex → biber → pdflatex → pdflatex`. La reproducibilidad es de extremo a extremo en el sentido fuerte: cualquier revisor puede regenerar el insumo bit a bit (huellas SHA-256 idénticas), reejecutar las doce series de medición y reconstruir cada cifra de este informe desde `resultados/tiempos_crudos.csv`, requisito que responde directamente a las debilidades de trazabilidad señaladas en la evaluación de la práctica anterior de la asignatura.

IV. RESULTADOS

IV-A. Tiempos medidos y speedup por transformación

La Tabla IV presenta las medianas de las cinco repeticiones por transformación para pandas y para PySpark con cuatro executors, junto con el speedup $S = T_{\text{pandas}}/T_{\text{spark}}$ y su

interpretación. El patrón es nítido: PySpark solo supera a pandas en T4 ($S = 1,2519$) y T5 ($S = 1,1244$), las dos transformaciones con mayor cómputo por registro (evaluación de la expresión de puntaje sobre 500 000 filas y ordenamiento global, respectivamente). En T1, T2 y T3 el speedup es inferior a la unidad, con el caso extremo de T2 ($S = 0,1704$): una agregación sobre solo 40 grupos cuyo cómputo útil es mínimo, de modo que el tiempo de PySpark queda dominado por la sobrecarga fija de planificación de tareas y shuffle.

La interpretación por transformación precisa el mecanismo en cada caso. En **T1**, un filtro con selección es una transformación estrecha sin shuffle; aun así PySpark tarda 0,3457 s frente a 0,2119 s de pandas, porque el costo de planificar y despachar las tasks a executors remotos y de recolectar el conteo supera al cómputo del predicado sobre 500 000 filas, que pandas resuelve vectorizado en un solo espacio de direcciones. En **T2**, la agrupación es ancha: exige un shuffle completo para reunir cada `laboratorio_id` en una partición, y como solo existen 40 claves el cómputo útil posterior es despreciable; el resultado es el peor speedup del experimento (0,1704), con casi todo el tiempo de PySpark imputable a la maquinaria y no al dato. En **T3**, el join también es ancho a este tamaño de configuración, y aunque la dimensión de 40 laboratorios es minúscula, el movimiento de las 500 000 filas de hechos por la red de shuffle domina el costo. En **T4**, en cambio, la columna derivada es estrecha y computacionalmente densa (conversiones temporales, cuatro bonos condicionales y un producto por fila), y es la única transformación en la que pandas supera los dos segundos; repartir ese cómputo entre cuatro executors compensa la sobrecarga y produce el mejor speedup (1,2519). En **T5**, el ordenamiento global es ancho pero el cómputo de comparación por fila es suficientemente costoso para que el ordenamiento parcial paralelo por partición, seguido de la fusión del top-20, supere ligeramente al ordenamiento estable monoproceso de pandas (1,1244).

Respecto de la dispersión, los tiempos crudos publicados en `resultados/tiempos_crudos.csv` muestran repeticiones estables: por ejemplo, las cinco repeticiones de T1 en pandas oscilaron entre 0,1884 s y 0,2539 s, y las de T3 con dos executors entre 0,6554 s y 0,9333 s; en todas las series la primera repetición medida tendió a ser de las más lentas, lo que confirma la pertinencia de haber descartado además una ejecución de calentamiento previa. El uso de la mediana como estadístico central neutraliza los valores extremos residuales de cada serie.

La Fig. 2 contrasta gráficamente ambos motores. Su lectura interpretativa es directa: las barras de PySpark superan a las de pandas precisamente en las transformaciones baratas por registro (T1–T3), donde el costo fijo del motor no se amortiza, y quedan por debajo en las costosas (T4–T5). Este comportamiento es coherente con la caracterización empírica de la literatura, según la cual la ventaja de Spark se manifiesta cuando el cómputo útil por partición excede con holgura la sobrecarga de coordinación [8], [5].

IV-B. Escalabilidad de T3 y fracción serial observada

Para habilitar el análisis de Amdahl, T3 se midió con 1, 2 y 4 executors. La Tabla V resume las medianas, el speedup

Tabla IV
MEDIANAS DE CINCO REPETICIONES POR TRANSFORMACIÓN (CALENTAMIENTO DESCARTADO) Y SPEEDUP PANDAS FRENTE A PYSARK CON 4 EXECUTORS.

Transformación	T_{pandas} (s)	T_{spark} (s)	Speedup S	Interpretación
T1 Filtrado compuesto	0,211 9	0,345 7	0,613 1	Sobrecarga de tareas supera al cómputo del filtro
T2 Agrupación (3 agreg.)	0,117 1	0,687 5	0,170 4	Shuffle domina; solo 40 grupos de salida
T3 Join N:1	0,288 9	0,865 7	0,333 7	Join con shuffle penalizado a este volumen
T4 Columna derivada	2,265 9	1,810 0	1,251 9	Cómputo por fila costoso: el paralelismo compensa
T5 Ordenamiento y top-20	1,525 2	1,356 5	1,124 4	Ordenamiento distribuido ligeramente favorable

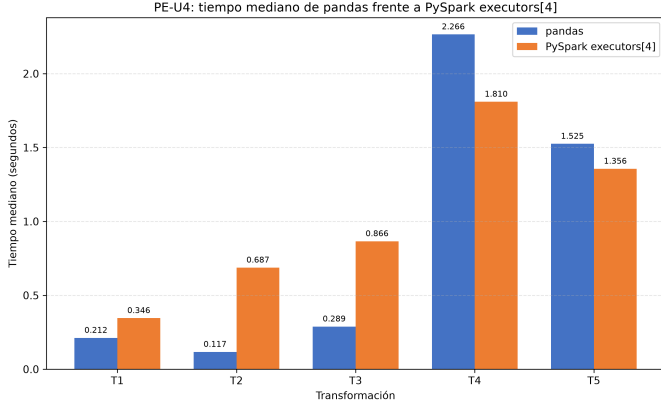


Figura 2. Tiempos medianos de pandas frente a PySpark (4 executors) por transformación. Generada con matplotlib a 300 DPI por src/graficas.py.

Tabla V
ESCALADO DE T3 (JOIN) CON 1, 2 Y 4 EXECUTORS: MEDIANA, SPEEDUP EXPERIMENTAL RESPECTO DE $N = 1$ Y EFICIENCIA SEGÚN (5).

N	T_N (s)	$S(N)$	$E(N)$ (%)
1	1,361 5	1,000 0	100,00
2	0,708 5	1,921 7	96,08
4	0,865 7	1,572 7	39,32

experimental respecto de la configuración con un executor, $S(N) = T_{\text{spark},1}/T_{\text{spark},N}$, y la eficiencia según (5).

El trabajo matemático se desarrolla de forma explícita. Con $N = 2$:

$$S(2) = \frac{T_{\text{spark},1}}{T_{\text{spark},2}} = \frac{1,361 5}{0,708 5} = 1,921 7, \quad (6)$$

y con $N = 4$:

$$S(4) = \frac{T_{\text{spark},1}}{T_{\text{spark},4}} = \frac{1,361 5}{0,865 7} = 1,572 7. \quad (7)$$

Sustituyendo la medición con $N = 4$ y $S = 1,572 7$ en (4), la fracción no escalable observada resulta:

$$\begin{aligned} f &= \frac{\frac{1}{1,572 7} - \frac{1}{4}}{1 - \frac{1}{4}} = \frac{0,635 9 - 0,250 0}{0,750 0} \\ &= \frac{0,385 9}{0,750 0} = 0,514 5, \end{aligned} \quad (8)$$

Tabla VI
SPEEDUP EXPERIMENTAL DE T3 FRENTE A LAS PREDICCIONES DE AMDAHL (1) Y GUSTAFSON-BARSIS (3) CON $f = 0,514 5$.

N	$S_{\text{exp}}(N)$	$S_A(N)$	$S_G(N)$
1	1,000 0	1,000 0	1,000 0
2	1,921 7	1,320 6	1,485 5
4	1,572 7	1,572 7	2,456 5

de donde la fracción paralelizable observada es $p = 1 - f = 0,485 5$. Aplicando (2), el límite teórico máximo de speedup para T3, con infinitos executors, es

$$S_{\text{máx}} = \frac{1}{1-p} = \frac{1}{f} = \frac{1}{0,514 5} = 1,943 7. \quad (9)$$

Con ese valor de p , el speedup teórico de Amdahl (1) predice

$$S_A(2) = \frac{1}{0,514 5 + \frac{0,485 5}{2}} = \frac{1}{0,757 2} = 1,320 6, \quad (10)$$

mientras que por construcción $S_A(4) = 1,572 7$ coincide con el punto experimental empleado en (8). Para el modelo escalado de Gustafson-Barsis (3):

$$S_G(2) = 2 - 0,514 5 (2 - 1) = 1,485 5, \quad (11)$$

$$S_G(4) = 4 - 0,514 5 (4 - 1) = 2,456 5. \quad (12)$$

La Tabla VI consolida el contraste entre los dos modelos teóricos y la medición. La lectura conjunta es reveladora: Amdahl subestima el punto $N = 2$ y, por construcción, coincide en $N = 4$; Gustafson, que responde a la pregunta distinta de cuánto más trabajo podría procesarse en el mismo tiempo, promete 2,456 5 con cuatro executors, valor inalcanzable en el experimento de talla fija pero indicativo del régimen al que el PFC debería aspirar aumentando el volumen por lote. La no monotonía se aprecia también en los tiempos absolutos de la Tabla V: el mínimo ocurre en $N = 2$ y el tramo de 2 a 4 executors tiene pendiente positiva, es decir, añadir recursos encareció el resultado.

La Fig. 3 superpone el speedup experimental de T3 sobre la curva teórica de Amdahl con $p = 0,485 5$ y su asíntota $S_{\text{máx}} = 1,943 7$. Su interpretación revela el hallazgo central del experimento: el punto experimental con $N = 2$ ($S = 1,921 7$) queda *por encima* de la predicción de Amdahl (10) y prácticamente sobre la asíntota, mientras que el punto con $N = 4$ cae sobre la curva por construcción. Es decir, el escalado real no es monótono: pasar de 2 a 4 executors *empeoró* el tiempo absoluto (0,708 5 frente a 0,865 7 s). Esta anomalía se discute en la Sección V.

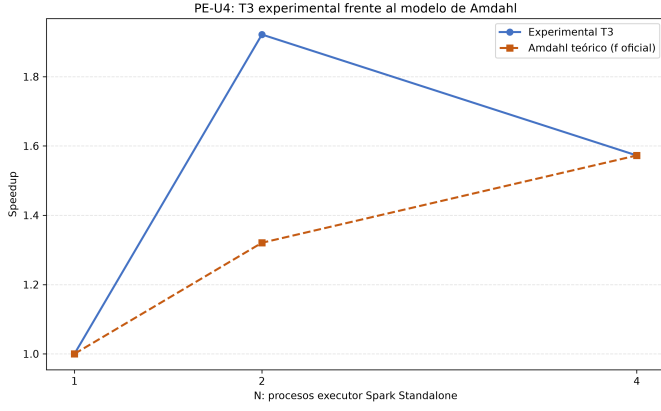


Figura 3. Speedup experimental de T3 frente al número de executors N , con la curva teórica de Amdahl (1) para $p = 0,4855$ y la asíntota $S_{\text{máx}} = 1,9437$ de (9) superpuestas. Generada con matplotlib a 300 DPI.

La Fig. 4 muestra la eficiencia (5) frente a N . Su lectura interpretativa cuantifica el desperdicio de recursos: con dos executors la eficiencia es casi ideal (96,08 %: el segundo executor aporta prácticamente todo su valor), pero con cuatro cae a 39,32 %, es decir, más del 60 % de la capacidad agregada se consume en sobrecarga de coordinación en lugar de cómputo útil. En términos de aprovisionamiento en la nube, pagar cuatro executors para este volumen equivale a desperdiciar la mayor parte de la factura, en línea con la motivación de los recomendadores de recursos conscientes del costo para flujos DAG [16]. La Fig. 5 completa el análisis contrastando ambos modelos teóricos: la recta de Gustafson (3) crece sin cota porque supone carga creciente, mientras que el experimento, de talla fija, sigue el régimen de Amdahl; la distancia creciente entre ambas curvas ilustra por qué cada modelo responde a una pregunta distinta [2], [3].

IV-C. Evidencia de ejecución: Spark UI

La Fig. 6 presenta la captura de la interfaz de Spark correspondiente al job de T3, en la que se aprecian el DAG del plan físico y los stages en que Catalyst y el planificador lo descomponen: un stage de lectura y proyección de cada fuente y el stage delimitado por la frontera de shuffle del join, con sus tasks distribuidas entre los executors registrados del clúster Standalone. La captura confirma visualmente dos afirmaciones del fundamento teórico: que el plan ejecutado no coincide literalmente con el orden textual del código, por la reescritura del optimizador [7], y que el shuffle del join es la frontera que parte el trabajo en stages [7]. El repositorio incluye además capturas del master con el worker y los executors registrados (evidencia/), que acreditan que la ejecución ocurrió sobre un clúster real y no sobre `local[N]`.

V. ANÁLISIS CRÍTICO

V-A. La anomalía $N = 2$ frente a $N = 4$

El resultado más instructivo del experimento es que T3 fue más rápida con dos executors (0,7085 s) que con cuatro (0,8657 s). La Ley de Amdahl (1) es monótona creciente en N , de modo que la anomalía no la explica el modelo, sino los

fig3_eficiencia.png

Figura 4. Eficiencia del paralelismo $E(N) = S(N)/N$ según (5) para T3 con 1, 2 y 4 executors. Generada con matplotlib a 300 DPI.

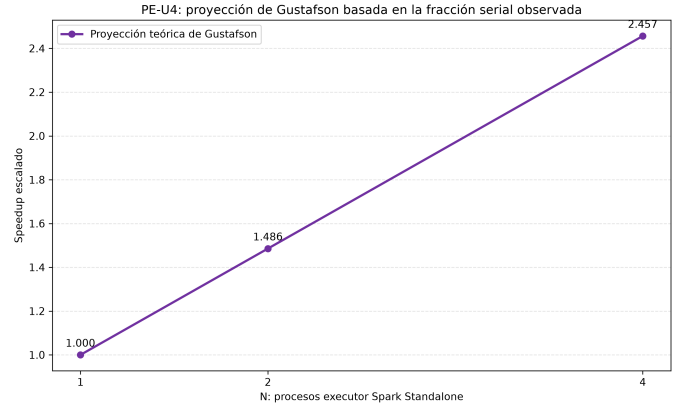


Figura 5. Speedup escalado de Gustafson-Barsis (3) para T3 con $f = 0,5145$, contrastado con el régimen de talla fija de Amdahl. Generada con matplotlib a 300 DPI.

términos que el modelo no incluye: la sobrecarga dependiente de N . Con `spark.sql.shuffle.partitions = N`, duplicar los executors duplica las particiones de shuffle del join; con 500 000 filas, las particiones resultantes son pequeñas y el costo por partición (planificación de la task, serialización, transferencia y apertura de flujos de shuffle) crece más de lo que decrece el cómputo útil por partición. En otras palabras, la sobrecarga no es constante como supone el modelo simple, sino creciente con N , y a este volumen su pendiente supera a la ganancia del paralelismo a partir de $N = 3$ aproximadamente. Este fenómeno está documentado en la literatura de benchmarking, donde la ventaja de añadir recursos depende críticamente de la relación entre volumen de datos y número de particiones [8], y es la razón por la que la métrica de Karp-Flatt recomienda examinar la evolución de f con N : si f crece con

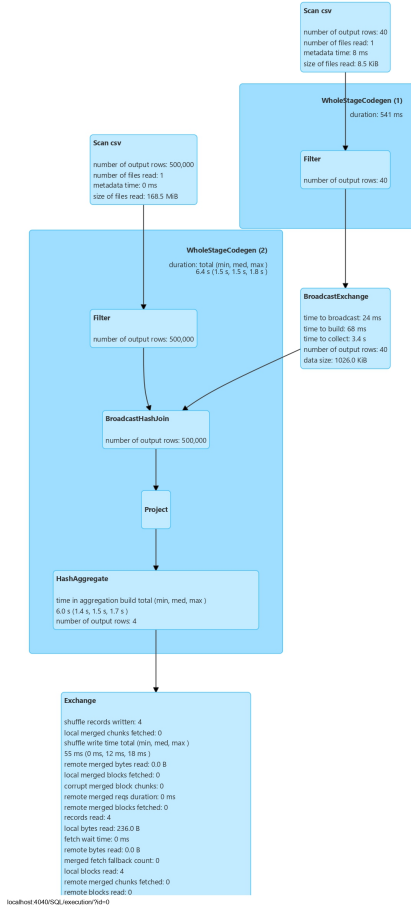


Figura 6. Captura de la Spark UI con el DAG y los stages del job de la transformación T3 (join) sobre el clúster Standalone. Archivo evidencia/spark_ui_t3.jpg del repositorio.

N , la pérdida de eficiencia se debe a sobrecarga y no a falta de paralelismo algorítmico [3]. En efecto, aplicando (4) al punto $N = 2$ se obtiene $f_2 = (1/1,9217 - 1/2)/(1 - 1/2) = 0,0407$, frente a $f_4 = 0,5145$ del punto $N = 4$: el crecimiento de f en un orden de magnitud al duplicar N es la firma inequívoca de la sobrecarga creciente del motor.

V-B. Fracción serial algorítmica frente a fracción no escalable observada

La discrepancia entre f_2 y f_4 obliga a la distinción conceptual exigida por la guía. La *fracción serial algorítmica* de un join particionado por clave es intrínsecamente pequeña: casi todo el trabajo (sondear la tabla dimensional y emitir filas combinadas) es paralelizable por partición. La *fracción no escalable observada* $f = 0,5145$ de (8) no mide esa propiedad del algoritmo, sino el agregado de la porción serial real (driver, planificación central, recolección del conteo) más toda la sobrecarga distribuida que no se reduce al añadir executors [3]. Por eso $S_{\max} = 1,9437$ de (9) debe leerse como el límite práctico de esta implementación a este volumen, no como una propiedad del join: con un conjunto de datos mayor, el cómputo útil crecería, la sobrecarga relativa caería,

f disminuiría y el límite se elevaría, que es exactamente el régimen de carga creciente que describe Gustafson (3) [2].

V-C. Umbral de rentabilidad de Spark en el dominio FUVV

La Tabla IV permite estimar el umbral a partir del cual Spark es rentable en el dominio. La sobrecarga fija observable del motor, tomando T2 como transformación de cómputo casi nulo, ronda los 0,5 s–0,7 s por acción a este volumen. Para que el paralelismo la amortice, el cómputo útil secuencial debe ser un múltiplo holgado de esa cifra: en el experimento, solo T4 ($T_{\text{pandas}} = 2,2659$ s) y T5 (1,5252 s) la superan, y en efecto son las únicas con $S > 1$. Extrapolando linealmente los tiempos por registro, las transformaciones baratas (T1–T3) requerirían del orden de $5\times$ a $20\times$ el volumen actual, es decir, entre 2,5 y 10 millones de solicitudes, para que Spark las rentabilice, cifra alcanzable en el dominio FUVV solo al consolidar varios períodos académicos o al incorporar la telemetría de monitoreo por equipo, mucho más voluminosa que las reservas. La conclusión operativa para el PFC es un diseño híbrido: pandas para los cortes analíticos periódicos del volumen actual y Spark reservado para el reprocesamiento histórico consolidado y para el flujo de monitoreo, decisión alineada con la recomendación de dimensionar recursos según el costo y no según el máximo disponible [16], [8].

V-D. Contraste con la evidencia publicada

Los resultados no contradicen la literatura que atribuye a Spark ventajas sobre los motores previos: la reconcilian con su dominio de validez. El estudio de benchmarking sistemático con HiBench compara Spark contra Hadoop sobre conjuntos de datos de decenas y centenas de gigabytes en clústeres de varios nodos, régimen en el que el cómputo útil por partición amortiza holgadamente la coordinación [8]; nuestro experimento compara Spark contra un motor monoproceso vectorizado sobre 500 000 filas que caben en memoria, régimen explícitamente señalado por la propia literatura del motor como territorio de las herramientas de un solo nodo [5]. Que T4 y T5 crucen ya el umbral de rentabilidad a este volumen, y que T1–T3 no lo hagan, dibuja empíricamente la frontera entre ambos regímenes y confirma la advertencia teórica de la unidad: la decisión entre procesamiento secuencial y distribuido no es ideológica sino cuantitativa, y debe tomarse midiendo, con protocolo, el caso propio [1], [3].

V-E. Implicaciones para el despliegue en la nube del PFC

Los resultados tienen traducción directa al dimensionamiento en la nube. Si el pipeline del sistema de laboratorios se desplegara sobre un servicio gestionado (EMR, HDInsight o Dataproc), la curva de eficiencia de la Fig. 4 sería literalmente la curva de retorno por dólar: con el volumen actual, un clúster de dos executors entrega el 96 % de eficiencia y uno de cuatro solo el 39 %, de modo que duplicar el gasto en cómputo empeoraría incluso el tiempo absoluto. La política racional, coherente con la literatura de recomendación de recursos consciente del costo para flujos DAG [16], es dimensionar por lote y no por pico: agrupar el procesamiento analítico en lotes

grandes (consolidando períodos o incorporando telemetría), aprovechar la elasticidad rápida de la nube [6] para crear el clúster solo durante el lote y destruirlo después, y elegir N con el criterio de eficiencia mínima de la Sección II-A en lugar del máximo presupuestable. El experimento aporta, además, el insumo que esa política necesita: la fracción no escalable medida por transformación, sin la cual cualquier dimensionamiento es conjetura.

V-F. Amenazas a la validez

Tres limitaciones acotan la generalización. Primera, el clúster Standalone se desplegó en una sola máquina física: los executors son procesos JVM independientes, pero comparten CPU, memoria y disco, de modo que la contención local infla la sobrecarga con $N = 4$ frente a un clúster de nodos físicos. Segunda, el conjunto de datos, aunque de 500 000 registros, cabe en la memoria de un solo proceso, que es precisamente el régimen donde pandas es imbatible; el experimento no observa el régimen en que pandas deja de poder ejecutar. Tercera, la mediana de cinco repeticiones es robusta pero no captura la varianza entre sesiones del clúster; los tiempos crudos publicados permiten a un revisor calcular dispersiones. Cuarta, cada executor se configuró con un solo núcleo y 512 MiB, valores modestos que acotan el paralelismo intra-executor y aumentan la presión de memoria durante el shuffle; con executors de varios núcleos la fracción no escalable sería previsiblemente menor. Quinta, el conjunto de datos es sintético y sus claves de agrupación están distribuidas de manera aproximadamente uniforme entre los 40 laboratorios, de modo que el experimento no observa el sesgo de partición (*data skew*) que en cargas reales suele ser la causa dominante de pérdida de eficiencia en transformaciones anchas.

Ninguna de estas amenazas invalida la comprobación de la ley: todas afectan el valor numérico de f , no la forma funcional (1) que el experimento confirma. La dirección del sesgo, además, es conocida y consistente: las cinco limitaciones tienden a sobrestimar f , por lo que el límite $S_{\max} = 1,9437$ debe leerse como una cota conservadora del rendimiento alcanzable en una infraestructura de producción.

VI. PREGUNTAS DE ANÁLISIS

VI-A. Pregunta 1: speedup teórico máximo de T3 y procesadores para el 90 %

Con el valor obtenido en (8), $f = 0,5145$ y $p = 0,4855$, el speedup teórico máximo para $N \rightarrow \infty$ ya se calculó en (9): $S_{\max} = 1/f = 1,9437$. El 90 % de ese máximo es $S_{90} = 0,9 \times 1,9437 = 1,7493$. Para hallar el número de procesadores que lo alcanza se iguala (1) a S_{90} y se despeja N :

$$\frac{1}{f + \frac{p}{N}} = S_{90} \implies N = \frac{p}{\frac{1}{S_{90}} - f}. \quad (13)$$

Sustituyendo numéricamente:

$$N = \frac{0,4855}{\frac{1}{1,7493} - 0,5145} = \frac{0,4855}{0,5717 - 0,5145} = \frac{0,4855}{0,0572} = 8,49. \quad (14)$$

Como N debe ser entero, se requieren **9 procesadores**. La comprobación con (1) lo confirma: $S(9) = 1/(0,5145 + 0,4855/9) = 1/0,5684 = 1,7592$, que es el 90,5 % de $S_{\max}$, mientras que $S(8) = 1/(0,5145 + 0,4855/8) = 1/0,5752 = 1,7386$ alcanza solo el 89,4 %. El resultado es pedagógicamente elocuente: nueve procesadores para no llegar siquiera a duplicar el rendimiento. Es la manifestación numérica del argumento original de Amdahl sobre los rendimientos decrecientes del paralelismo cuando la fracción serial es alta [1], y del uso diagnóstico de la fracción serial experimental que proponen Karp y Flatt: cuando f supera 0,5, invertir en más procesadores es económicamente indefendible y el esfuerzo debe dirigirse a reducir la sobrecarga, no a escalar [3]. Para el dominio FUVV, la implicación directa es que el join de solicitudes con laboratorios no justifica, al volumen actual, más de dos executors, y que todo presupuesto adicional rendiría más invertido en aumentar el volumen procesado por lote, régimen en el que gobierna Gustafson [2].

VI-B. Pregunta 2: speedup menor que 1, sobrecarga de Spark y tamaño mínimo del conjunto

Tres de las cinco transformaciones (T1, T2 y T3) presentaron speedup inferior a la unidad, con PySpark hasta seis veces más lento que pandas en T2. La causa no es un defecto de implementación, verificada como equivalente en la Tabla III, sino la estructura de costos del motor distribuido. Cada acción de Spark paga una sobrecarga que pandas no tiene: construcción y optimización del plan por Catalyst [7], planificación y despacho de tasks a executors remotos, serialización y deserialización de datos entre JVM y, en las transformaciones anchas como la agrupación de T2 y el join de T3, el shuffle completo con escritura y lectura de bloques intermedios [7], [5]. En el experimento esa sobrecarga se cuantifica directamente: T2 computa tres agregados sobre apenas 40 grupos, un trabajo que pandas resuelve en 0,1171 s, mientras PySpark tarda 0,6875 s; la diferencia, del orden de 0,5 s–0,6 s, es esencialmente sobrecarga pura de planificación y shuffle a este volumen. El tamaño mínimo que justifica Spark en el dominio FUVV se estima exigiendo que el cómputo útil secuencial supere con holgura esa sobrecarga multiplicada por el factor de amortización deseado: extrapolando los tiempos por registro medidos, las transformaciones baratas necesitarían entre 2,5 y 10 millones de registros, volumen que en el PFC solo se alcanza consolidando varios períodos académicos o incorporando la telemetría de monitoreo por equipo. Esta estimación coincide con la evidencia empírica publicada, que sitúa la ventaja de Spark en regímenes donde el volumen por partición mantiene ocupados los executors [8], y con la advertencia general de que los datos pequeños pertenecen a herramientas de un solo nodo [7].

VI-C. Pregunta 3: pipeline del PFC con Spark Structured Streaming

Si las solicitudes de reserva y los eventos de monitoreo del sistema FUVV llegan en tiempo real, el pipeline se reimplentaría sobre Spark Structured Streaming siguiendo una arquitectura Kappa [17], [14]. Como *fuentes*, cada microservicio

del sistema publicaría sus eventos (solicitud creada, aprobada, cancelada; equipo encendido, incidencia) en tópicos de un log distribuido tipo Kafka, particionados por `laboratorio_id` para preservar el orden por laboratorio; el job de streaming leería esos tópicos como DataFrames ilimitados. Las transformaciones T1–T5 se trasladan casi literalmente gracias a que la API de DataFrames unifica lote y flujo [7], [5]: T1 es un filtro sin estado; T2 se convierte en agregaciones por ventanas de tiempo deslizantes por laboratorio con `watermark` para acotar el estado y tolerar eventos tardíos; T3 es un *stream-static join* contra la dimensión de laboratorios, pequeña y difundida a los executors; T4 sigue siendo una columna derivada sin estado; y T5 se materializa como vista de ranking actualizada por ventana. Como *sinks*, los agregados de demanda y el ranking se escribirían en la base operacional del dashboard mediante `foreachBatch`, y el histórico crudo en almacenamiento columnar para reprocesamiento. La *política de triggers* sería de micro-lotes por tiempo de procesamiento (por ejemplo, cada 30 segundos), plazo sobrado para un tablero administrativo y que reduce la sobrecarga por lote que este mismo experimento cuantificó. Las *garantías de entrega* serían exactamente-una-vez de extremo a extremo, combinando fuentes reproducibles por offset, checkpointing del estado y del progreso en almacenamiento confiable, y sinks idempotentes o transaccionales [17], [5]; donde el sink solo admita al-menos-una-vez, la idempotencia por clave de solicitud preservaría la corrección.

VI-D. Pregunta 4: eficiencia de Spark frente a MapReduce en procesamiento iterativo

La ventaja de Spark sobre MapReduce en cargas iterativas se explica por la conjunción de dos mecanismos: el *lineage* de los RDD y la *evaluación perezosa*. En MapReduce, cada iteración de un algoritmo es un trabajo independiente que lee su entrada de HDFS y escribe su salida en HDFS, con replicación incluida; la tolerancia a fallos se compra con materialización a disco en cada frontera, de modo que un algoritmo de k iteraciones paga k ciclos completos de E/S distribuida [4], [9]. El RDD invierte ese compromiso: los datos de trabajo permanecen en la memoria de los executors entre iteraciones, y la tolerancia a fallos no requiere replicarlos porque cada RDD registra su lineage, el grafo de transformaciones gruesas que lo produjo; si se pierde una partición, el motor la reconstruye reaplicando ese linaje solo sobre los datos necesarios, un costo que se paga únicamente ante el fallo y no en cada iteración [5]. La evaluación perezosa complementa el mecanismo: al no ejecutar nada hasta la acción, Spark acumula la cadena completa de transformaciones de la iteración y la compila en un DAG de stages que encadena operaciones estrechas dentro de un mismo stage sin materializar intermedios, dejando el shuffle solo donde es semánticamente inevitable [5], [7]; sobre DataFrames, Catalyst además reescribe el plan global antes de generarlo físicamente [7]. El resultado, validado empíricamente por estudios de benchmarking sistemático con ventajas especialmente amplias en cargas iterativas de aprendizaje automático [8], es que el costo por iteración de Spark se reduce al cómputo más el shuffle estrictamente necesario, mientras que el de MapReduce incluye siempre la materialización replicada

a disco. Nuestro propio experimento ofrece la contracara pedagógica: en trabajos de una sola pasada y volumen modesto, esas mismas ventajas no alcanzan a amortizar la sobrecarga fija del motor.

VII. CONCLUSIONES INDIVIDUALES

Como síntesis grupal previa a las conclusiones individuales, el equipo considera comprobada experimentalmente la Ley de Amdahl en su doble dimensión: como forma funcional, porque el speedup medido de T3 satura en lugar de crecer linealmente con los executors, y como herramienta de ingeniería, porque su forma inversa permitió cuantificar en $f = 0,5145$ la fracción no escalable de la implementación y en $S_{\max} = 1,9437$ su límite práctico. El hallazgo no previsto, el mejor rendimiento con dos executors que con cuatro, resultó ser el más valioso pedagógicamente, porque obligó a distinguir entre el modelo y los términos de sobrecarga que el modelo omite, y a usar la métrica de Karp-Flatt como diagnóstico y no solo como fórmula. A continuación, cada integrante presenta su conclusión individual.

VII-A. Freddy Farinango

Coordinar esta práctica y levantar el clúster Spark Standalone real me dejó un aprendizaje técnico que ningún ejercicio con `local[N]` habría producido: la diferencia entre simular paralelismo e instrumentarlo. Exigir mediante el StatusTracker que los executors estuvieran registrados antes de medir, fijar las particiones de shuffle al número de executors y verificar que el master no degradara a modo local convirtieron la configuración del motor en parte del experimento y no en un trámite. El resultado que más me hizo pensar fue la anomalía de T3: dos executors rindieron más que cuatro, y la fracción de Karp-Flatt saltó de 0,0407 a 0,5145 al duplicar N , evidencia numérica de que la sobrecarga del motor crece con los recursos. Para el PFC de laboratorios informáticos, del que provengo, la lección es directa: nuestro sistema de reservas no debe adoptar Spark por moda, sino cuando el volumen consolidado de telemetría lo amortice, y cualquier decisión de aprovisionamiento debe partir de una medición con protocolo, calentamiento descartado y mediana, como la que aquí construimos.

VII-B. Jeremy Gaibor

Implementar dos veces las mismas cinco transformaciones, primero en pandas y luego en PySpark, me enseñó que la equivalencia semántica entre motores es un problema de ingeniería en sí mismo. Detalles que parecían menores decidían la validez del experimento: reproducir en Spark la normalización UTC de fechas de pandas extrayendo la subcadena ISO para que el bono de anticipación de T4 coincidiera, recordar que `dayofweek` numera el domingo como 1 en Spark y como 6 en pandas, y garantizar el desempate estable del top-20 con cuatro claves de ordenación. Sin la verificación de cardinalidades y agregados de control, cualquier speedup habría sido ruido. También interioricé el peso de la evaluación perezosa: hasta que no forzamos la acción `count()`, los

tiempos de PySpark medían la construcción del plan y no su ejecución, un error que habría inflado artificialmente al motor distribuido. Aplicado al dominio FUVV, concluyo que el puntaje de prioridad de T4, la única transformación donde Spark ganó con claridad, es exactamente el tipo de cómputo por registro que justificaría distribuir el pipeline cuando el sistema crezca.

VII-C. Iván Villamarín

Mi rol de análisis cuantitativo me obligó a usar las ecuaciones de la unidad como herramientas y no como fórmulas de examen, y ahí estuvo mi mayor aprendizaje técnico. Sustituir las medianas reales en la forma inversa de Amdahl y obtener $f = 0,5145$ me pareció primero un fracaso del experimento, hasta comprender la advertencia metodológica de la guía: esa cifra no es la fracción serial del algoritmo de join, sino la fracción no escalable observada, que arrastra toda la sobrecarga de planificación y shuffle del motor. Distinguir las cambia la conclusión por completo: el límite $S_{\text{máx}} = 1,9437$ describe esta implementación a este volumen, no el potencial del join. Calcular que se necesitarían nueve procesadores para rozar el 90 % de ese límite, y contrastarlo con la recta de Gustafson que promete 2,4565 con solo cuatro si la carga creciera, me dejó clara la pregunta correcta para el PFC de referencia: no cuántos executors comprar, sino cuánto volumen procesar por lote para que los executors que ya tenemos trabajen con eficiencia superior al 90 %, como ocurrió con $N = 2$.

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

En cumplimiento de los lineamientos de la asignatura, el equipo declara el uso de la herramienta **Claude (Anthropic)** como recurso de apoyo durante el desarrollo de la actividad. Su utilización estuvo orientada principalmente a aclarar dudas, comprender conceptos relacionados con los temas abordados, interpretar determinadas indicaciones de la actividad y recibir orientación cuando algún procedimiento o concepto requiera una explicación adicional. También se utilizó de manera puntual para revisar errores ortográficos, gramaticales y mejorar la claridad de algunas expresiones. La herramienta no fue empleada para desarrollar el programa, implementar las transformaciones, realizar las mediciones, generar los resultados experimentales ni efectuar los cálculos presentados en el informe. El código, las decisiones técnicas, la ejecución del experimento, el análisis de los resultados y la revisión final del contenido fueron realizados por los integrantes del equipo, quienes asumen la responsabilidad sobre el trabajo presentado.

REFERENCIAS

- [1] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.
- [2] J. L. Gustafson, «Reevaluating Amdahl's Law,» *Communications of the ACM*, vol. 31, n.º 5, págs. 532-533, 1988. DOI: 10.1145/42411.42415.
- [3] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614.
- [4] J. Dean y S. Ghemawat, «MapReduce: Simplified Data Processing on Large Clusters,» *Communications of the ACM*, vol. 51, n.º 1, págs. 107-113, 2008. DOI: 10.1145/1327452.1327492.
- [5] M. Zaharia et al., «Apache Spark: A Unified Engine for Big Data Processing,» *Communications of the ACM*, vol. 59, n.º 11, págs. 56-65, 2016. DOI: 10.1145/2934664.
- [6] P. Mell y T. Grance, «The NIST Definition of Cloud Computing,» National Institute of Standards y Technology, Gaithersburg, MD, USA, Special Publication 800-145, 2011. DOI: 10.6028/NIST.SP.800-145.
- [7] M. Armbrust et al., «Spark SQL: Relational Data Processing in Spark,» en *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*, Melbourne, VIC, Australia: ACM, 2015, págs. 1383-1394. DOI: 10.1145/2723372.2742797.
- [8] N. Ahmed, A. L. C. Barczak, T. Susnjak y M. A. Rashid, «A Comprehensive Performance Analysis of Apache Hadoop and Apache Spark for Large Scale Data Sets Using HiBench,» *Journal of Big Data*, vol. 7, n.º 1, pág. 110, 2020. DOI: 10.1186/s40537-020-00388-5.
- [9] K. Shvachko, H. Kuang, S. Radia y R. Chansler, «The Hadoop Distributed File System,» en *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, Incline Village, NV, USA: IEEE, 2010, págs. 1-10. DOI: 10.1109/MSST.2010.5496972.
- [10] T. White, *Hadoop: The Definitive Guide*, 4.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2015, ISBN: 978-1-4919-0163-2.
- [11] V. K. Vavilapalli et al., «Apache Hadoop YARN: Yet Another Resource Negotiator,» en *Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC '13)*, Santa Clara, CA, USA: Association for Computing Machinery, 2013, 5:1-5:16. DOI: 10.1145/2523616.2523633.
- [12] M. Zaharia et al., «Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,» en *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, San Jose, CA, USA: USENIX Association, 2012, págs. 15-28.
- [13] B. Chambers y M. Zaharia, *Spark: The Definitive Guide. Big Data Processing Made Simple*. Sebastopol, CA, USA: O'Reilly Media, 2018, ISBN: 978-1-4920-3251-9.
- [14] M. Armbrust et al., «A View of Cloud Computing,» *Communications of the ACM*, vol. 53, n.º 4, págs. 50-58, 2010. DOI: 10.1145/1721654.1721672.
- [15] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Systems Using Kuber-*

netes, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2024, ISBN: 978-1-098-15635-0.

- [16] M.-M. Aseman-Manzar, S. Karimian-Aliabadi, R. Entezari-Maleki, B. Egger y A. Movaghar, «Cost-Aware Resource Recommendation for DAG-Based Big Data Workflows: An Apache Spark Case Study,» *IEEE Transactions on Services Computing*, vol. 16, n.º 3, págs. 1726-1737, 2023. DOI: 10.1109/TSC.2022.3203010.
- [17] M. Armbrust et al., «Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark,» en *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*, Houston, TX, USA: Association for Computing Machinery, 2018, págs. 601-613. DOI: 10.1145/3183713.3190664.